

Chapter 10

Quantization Techniques

Exam Study Guide, Worked Concepts, Sample Questions, and

Prepared from	uploaded course materials
Main source	Chapter_10_Quantization.ipynb
Exam source	Chapter_10_Summary_and_Sample_Questions.pdf
Practice source	Rapid-review questions generated from the uploaded material

How to use it

Study quantization as controlled numerical approximation. The main question is where, when, and how the model absorbs low-precision error while preserving function.

Study order: First read the concept sections, then solve the five sample exam questions, then test yourself with the flashcards.

Contents

- 1. Quick exam map
- 2. Quantization as approximation
- 3. Uniform quantization and scale-range tradeoff
- 4. Granularity and what to quantize
- 5. Calibration and sensitivity
- 6. Quantization-aware training
- 7. Post-training quantization methods
- 8. Generative and multimodal issues
- 9. Formula cheat sheet and symbol key
- 10. Code prompts
- 11. Exam-style questions
- 12. Flashcards

Most useful mental shortcut
Quantization trades precision for deployment efficiency. The central battle is rounding error versus clipping error.

1. Quick exam map

Quantization reduces numerical precision so large models can run with lower memory, bandwidth, latency, and energy.

Full precision

model

FP32/FP16

weights and activations

Calibration

measure ranges and outliers

Quantizer design

bit-width and scale granularity

PTQ or QAT

compress model

Deployment

check accuracy, speed, memory, failure modes Figure: the main method pipeline for this chapter.

Exam topic	What you must be able to say
Why quantize?	Reduce memory, bandwidth, latency, and energy.

Exam topic	What you must be able to say
Uniform quantizer	Scale determines grid spacing and representable range.
Granularity	Per-tensor, per-channel, and group-wise scales.
QAT	Train under fake quantization with surrogate gradients.
PTQ	Quantize a pretrained model using calibration/reconstruction.
Generative issues	Outliers, residual propagation, KV cache, diffusion timestep sensitivity, multimodal geometry.

2. Zero-prior explanation: understand the

idea before the equations This section assumes you have not read the lecture yet. The goal is to build intuition first, then connect the intuition to the equations and exam language.

Quantization as approximation

Quantization maps real values to a small set of integer codes. It is not just rounding; it is a design problem involving scale, range, granularity, calibration, and sensitivity.

Scale-range tradeoff

A small scale gives fine resolution but clips outliers. A large scale covers outliers but makes normal values coarse. Good quantization balances these errors.

What and how to quantize

Weights, activations, and KV caches can be quantized. Per-channel and group-wise scaling often work better than per-tensor scaling because they adapt to tensor structure.

Memory line

Quantization is the art of deciding where the model can afford numerical error.

3. Core concepts and worked explanations

Calibration and Hessian sensitivity

Calibration estimates activation ranges and outliers. Hessian-based methods estimate which weight perturbations are most harmful, explaining why GPTQ can beat naive rounding.

QAT

Quantization-aware training inserts fake quantization during training so the model learns to tolerate low precision. The straight-through estimator supplies approximate gradients through rounding.

PTQ and LLM methods

RTN is simple rounding. AdaRound optimizes rounding. GPTQ uses curvature compensation.

SmoothQuant migrates activation difficulty into weights. AWQ protects important channels.

OmniQuant learns clipping and equivalent transformations.

Generative and multimodal models

Autoregressive LLMs suffer from outliers and KV-cache accumulation. Diffusion errors compound across timesteps. VLMs must preserve image-text geometry and cross-modal interactions.

4. Method explanation, worked example,

and application

Step	Method explanation
Measure ranges	Use calibration data to estimate outliers and sensitivity.
Choose design	Select bit width, scale granularity, and mixed precision.
Apply PTQ/QAT	Compress pretrained model or train with fake quantization.
Validate deployment	Check task quality, latency, memory, and failure modes.

Worked example

Example: converting FP16 weights to INT8 reduces memory and bandwidth, but a bad scale can either clip outliers or make the grid too coarse for most values.

Applications and usage

Usage: quantization enables LLM/VLM deployment on limited GPUs, edge devices, mobile hardware, and production systems where latency, memory, and energy matter.

5. Formula / definition cheat sheet

Read formulas as compact sentences. Do not memorize symbols blindly; use the symbol key table after the formula table to understand what each part represents.

Formula	Interpretation
$q=\text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$ Uniform quantization.	$\hat{x}=s\cdot q$ Dequantization.
$\text{error} = \text{rounding error} + \text{clipping error}$ Main distortion sources.	$\Delta L \approx 1/2 \delta^T H \delta$ Second-order loss damage.
$\min_{\theta} L(Q(W\theta), Q(a\theta))$ Training under quantized behavior.	$\partial \text{round}(x)/\partial x \approx 1$ Straight-through estimator.

$XW=(XS)(S^{-1}W)$ Equivalent rescaling such as SmoothQuant.

6. Equation symbol key table

This table explains the main symbols used in the equations. When studying, cover the meaning column and check whether you can explain each symbol clearly.

Symbol	Meaning	Plain-English reading
x	Original real value	Full-precision number.

Symbol	Meaning	Plain-English reading
q	Integer code	Quantized representation.
s	Scale/step size	Converts between real values and
integer grid.	qmin,qmax	Integer limits
Lowest/highest available quantized	code.	$\hat{x}$
Dequantized value	Approximate real value reconstructed	from q.
δ	Perturbation	Quantization-induced parameter
change.	H	Hessian
Curvature/sensitivity matrix.	Q($\cdot$)	Quantizer
Operation mapping values to low	precision.	S
Scaling transform	Equivalent rescaling used to improve	quantization.

7. Main code prompt / implementation

skeleton This is not meant to replace the lecture code. It is a compact prompt showing the computational pattern you should recognize in homework and exams.

```
# Simple symmetric quantization prompt
scale = max(abs(x)) / qmax
q = clip(round(x / scale), -qmax, qmax)
x_dequant = scale * q
# Calibration prompt
for batch in calibration_loader:
    record_activation_ranges(model, batch)
```

How to explain the code in words
Start by identifying the input, the model output, the objective being minimized, and the update rule.
Then connect the code back to the formulas and the method table.

8. Common exam traps

Trap	Correct way to think
Thinking quantization is just	rounding It also involves scale, clipping, granularity, calibration, and sensitivity.
Ignoring activation outliers	Outliers can dominate scale and destroy precision.
Claiming full deployment from	weight-only quantization Activations, nonlinear ops, KV cache, and full graph behavior matter.

9. Exam-style sample questions with

answers These are written in the style of conceptual exam answers. Use them to practice explaining, not only recognizing terms.

Question 1

What is the scale-range tradeoff?

Sample answer

The scale controls both level spacing and representable range, creating a tension between rounding and clipping.

Question 2

Why is per-tensor scaling sometimes weak?

Sample answer

One outlier can dominate the scale and waste precision for common values.

Question 3

Why does QAT need STE?

Sample answer

Rounding is nondifferentiable, so training uses a surrogate gradient.

Question 4

How does GPTQ use curvature?

Sample answer

It estimates harmful perturbations and compensates remaining weights.

Question 5

Why are generative models hard to quantize?

Sample answer

Errors can accumulate over long contexts, denoising steps, or multimodal interactions.

10. Flashcards for rapid review

Use these for active recall. Cover the answer column and try to answer before checking.

Card 1

Q: Why quantize?

A: Efficiency: memory, latency, bandwidth, energy.

Card 2

Q: What is rounding error?

A: Error from coarse grid levels.

Card 3

Q: What is clipping error?

A: Error from values outside range.

Card 4

Q: What is PTQ?

A: Post-training quantization.

Card 5

Q: What is QAT?

A: Training with fake quantization.

Card 6

Q: What is SmoothQuant?

A: Equivalent rescaling to handle activation outliers.

Card 7

Q: What is AWQ?

A: Activation-aware weight quantization.

Card 8

Q: What is KV cache?

A: Stored attention keys/values during decoding.

Card 9

Q: What must you remember about Why quantize??

A: Reduce memory, bandwidth, latency, and energy.

Card 10

Q: What must you remember about Uniform quantizer?

A: Scale determines grid spacing and representable range.

Card 11

Q: What must you remember about Granularity?

A: Per-tensor, per-channel, and group-wise scales.

Card 12

Q: What must you remember about QAT?

A: Train under fake quantization with surrogate gradients.

Card 13

Q: What must you remember about PTQ?

A: Quantize a pretrained model using calibration/reconstruction.

Card 14

Q: What must you remember about Generative issues?

A: Outliers, residual propagation, KV cache, diffusion timestep sensitivity, multimodal geometry.

Card 15

Q: What does this formula represent:

$q = \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})?$

A: Uniform quantization.

Card 16

Q: What does this formula represent: $\hat{x} = s \cdot q$?

A: Dequantization.

Card 17

Q: What does this formula represent: error = rounding error + clipping error?

A: Main distortion sources.

Card 18

Q: What does this formula represent: $\Delta L \approx 1/2 \delta^T H \delta$?

A: Second-order loss damage.

Card 19

Q: What does this formula represent: $\min_{\theta} L(Q(W\theta), Q(a\theta))$?

A: Training under quantized behavior.

Card 20

Q: What does this formula represent:

$\partial \text{round}(x) / \partial x \approx 1$?

A: Straight-through estimator.

Card 21

Q: What does this formula represent:

$XW = (XS)(S^{-1}W)$?

A: Equivalent rescaling such as SmoothQuant.

Card 22

Q: What does x mean?

A: Original real value. Full-precision number.

Card 23

Q: What does q mean?

A: Integer code. Quantized representation.

Card 24

Q: What does s mean?

A: Scale/step size. Converts between real values and integer grid.

Card 25

Q: What does $q_{\min}, q_{\max}$ mean?

A: Integer limits. Lowest/highest available quantized code.

Card 26

Q: What does $\hat{x}$ mean?

A: Dequantized value. Approximate real value reconstructed from q .

Card 27

Q: What does δ mean?

A: Perturbation. Quantization-induced parameter change.

Card 28

Q: What does H mean?

A: Hessian. Curvature/sensitivity matrix.

Card 29

Q: What does $Q(\cdot)$ mean?

A: Quantizer. Operation mapping values to low precision.

Card 30

Q: What is the key idea of Quantization as approximation?

A: Quantization maps real values to a small set of integer codes. It is not just rounding; it is a design problem involving scale, range, granularity, calibration, and

Card 31

Q: What is the key idea of Scale-range tradeoff?

A: A small scale gives fine resolution but clips outliers. A large scale covers outliers but makes normal values coarse. Good quantization balances these

Card 32

Q: What is the key idea of What and how to quantize?

A: Weights, activations, and KV caches can be quantized. Per-channel and group-wise scaling often work better than per-tensor scaling because they adapt to tensor

Card 33

Q: What is the key idea of Calibration and Hessian sensitivity?

A: Calibration estimates activation ranges and outliers. Hessian-based methods estimate which weight perturbations are most harmful, explaining why GPTQ can beat naive

Card 34

Q: What is the key idea of QAT?

A: Quantization-aware training inserts fake quantization during training so the model learns to tolerate low precision. The straight-through estimator supplies approximate gradients through

Card 35

Q: What is the key idea of PTQ and LLM methods?

A: RTN is simple rounding. AdaRound optimizes rounding. GPTQ uses curvature compensation. SmoothQuant migrates activation difficulty into weights. AWQ protects important channels.

OmniQuant learns clipping and equivalent.

Card 36

Q: What is the key idea of Generative and multimodal models?

A: Autoregressive LLMs suffer from outliers and KV-cache accumulation. Diffusion errors compound across timesteps. VLMs must preserve image-text geometry and cross-modal

Card 37

Q: What happens in the step: Measure ranges?

A: Use calibration data to estimate outliers and sensitivity.

Card 38

Q: What happens in the step: Choose design?

A: Select bit width, scale granularity, and mixed precision.

Card 39

Q: What happens in the step: Apply PTQ/QAT?

A: Compress pretrained model or train with fake quantization.

Card 40

Q: What happens in the step: Validate deployment?

A: Check task quality, latency, memory, and failure modes.

Card 41

Q: Can you explain quantization beyond rounding?

A: Scale, clipping, granularity, calibration, sensitivity.

Card 42

Q: Can you choose PTQ vs QAT?

A: PTQ cheap; QAT more robust but expensive.

Card 43

Q: Can you name LLM issues?

A: Outliers, residuals, attention, KV cache.

Card 44

Q: Can you explain why VLMs are fragile?

A: Must preserve shared image-text geometry.

Card 45

Q: What is the scale-range tradeoff?

A: The scale controls both level spacing and representable range, creating a tension between rounding and clipping.

Card 46

Q: Why is per-tensor scaling sometimes weak?

A: One outlier can dominate the scale and waste precision for common values.

Card 47

Q: Why does QAT need STE?

A: Rounding is nondifferentiable, so training uses a surrogate gradient.

Card 48

Q: How does GPTQ use curvature?

A: It estimates harmful perturbations and compensates remaining weights.

Card 49

Q: Why are generative models hard to quantize?

A: Errors can accumulate over long contexts, denoising steps, or multimodal interactions.

Card 50

Q: In plain English, why is $q = \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$ important?

A: It captures: Uniform quantization. Use it as a compact exam sentence, then define each symbol.

Card 51

Q: In plain English, why is $\hat{x} = s \cdot q$ important?

A: It captures: Dequantization. Use it as a compact exam sentence, then define each symbol.

Card 52

Q: In plain English, why is $\text{error} = \text{rounding error} + \text{clipping error}$ important?

A: It captures: Main distortion sources. Use it as a compact exam sentence, then define each symbol.

Card 53

Q: In plain English, why is $\Delta L \approx 1/2 \delta^T H \delta$ important?

A: It captures: Second-order loss damage. Use it as a compact exam sentence, then define each symbol.

Card 54

Q: In plain English, why is $\min_{\theta} L(Q(W\theta), Q(a\theta))$ important?

A: It captures: Training under quantized behavior.

Use it as a compact exam sentence, then define each symbol.

Card 55

Q: In plain English, why is $\partial \text{round}(x) / \partial x \approx 1$ important?

A: It captures: Straight-through estimator. Use it as a compact exam sentence, then define each symbol.

Explanation

Chapter 10: Quantization Techniques

How this section is organized

Each item below is a heading taken from the corresponding Jupyter notebook in the same order it appears there. The explanation beside it tells you what that notebook section is trying to teach and why it matters for the review.

Source notebook

Chapter_10_Quantization.ipynb

Headings included

76

Notebook headings with explanation

1. Chapter 10 - Quantization Techniques for Deep Learning and Generative Models

This notebook heading introduces chapter 10 - quantization techniques for deep learning and generative models and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.

2. Learning Outcomes

By the end of this chapter, the learner should be able to: 1. Explain the motivation for quantization in terms of memory reduction, faster inference, lower bandwidth, lower energy use, and practical deployment.

3. Part I. Problem formulation and historical timeline

This notebook heading introduces part i. problem formulation and historical timeline and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.

4. Why do quantization?

Modern neural networks are often trained in FP32, BF16, or FP16, but deployment is limited by three coupled bottlenecks: - Memory footprint - Memory bandwidth - Arithmetic throughput Suppose a model has parameters $W = W_{\text{ell_ell}} = 1^L$. If a layer has n_{ell} scalar weights stored in b bits, then the storage for that layer is $M_{\text{ell}} = n_{\text{ell}} b$ bits.

5. The quantization problem as constrained approximation

Let $f(x; W)$ be a trained network and let $Q(W)$ denote the quantized parameters. The basic approximation problem is $Q(W) = \operatorname{argmin}_{\widehat{W} \in Q} D(W, \widehat{W})$, where Q is the discrete set of representable quantized values and D is some distortion measure.

6. Historical timeline

A compact research timeline is: - 2015 to 2016: binary and ternary ideas become influential, including BinaryConnect and XNOR-Net - 2016 to 2018: low-bit CNN training becomes more systematic, including DoReFa-Net and PACT - 2018: practical 8-bit deployment becomes mainstream with integer arithmetic pipelines and affine quantization formulations - 2019 to 2020: learned quantizers and stronger PTQ emerge, such as LSQ and AdaRound - 2021 onward: transformers and integer-only transformer inference become.

7. Notation guide

This chapter uses a common notation across all three phases: pre-quantization, during-training quantization, and post-training quantization. Scalars, vectors, matrices, and tensors - A scalar is written as x , s , α , or Δ . - A vector is written in bold lower case, for example w in $\mathbb{R}^n$. - A matrix is written in bold upper case, for example W in $\mathbb{R}^m \times n$. - A tensor is written with an explicit shape when useful, for example X in $\mathbb{R}^B \times C \times H \times W$. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

8. Three Phases of Quantization

Quantization studies how to replace high-precision numerical representations with lower-precision ones while preserving the functional behavior of a model as much as possible. The motivation is both practical and scientific.

9. How the methods are related	This chapter is organized around three phases, but they all revolve around the same core quantizer that takes an original parameter (say weight) x , scales it using s , uses an optional offset z and ensures that the resulting quantized quantity q is within the physical limits of $q_{\min}$ and $q_{\max}$ $q = \text{clip}(\text{round}(\text{left}(x/\text{sright}) + z,; q_{\min},; q_{\max}\text{right}),; x_{\text{hat}} = s (q - z)$. The difference is when and how we choose or optimize the quantization parameters. $x=10$ and $s=2.5$ $x=10$ and $s=0.25$ Pre-quantization.
10. Part II. Core mathematical foundations	This notebook heading introduces part ii. core mathematical foundations and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
11. Uniform affine quantization	A standard affine quantizer maps a real scalar x to an integer q through a scale $s > 0$ and zero point z . The quantization rule is $q = \text{clip}(\text{round}(x/s) + z,; q_{\min},; q_{\max})$.
12. Motivation: the tradeoff between scale and range in quantization	A central design question in quantization is the choice of the scale. What the scale controls The integer code is $q = \text{clip}(\text{round}(\text{left}(x/\text{sright}), q_{\min}, q_{\max}\text{right}),$ so the representable real-valued levels are $s q_{\min},; s(q_{\min}+1),; \text{dots},; s q_{\max}$.
13. Symmetric quantization	In symmetric quantization we set the zero point to zero: $z = 0$. The reconstruction is $x_{\text{hat}} = s q$.
14. Quantization error and clipping	If x is quantized to x_{hat} , then the scalar quantization error is $e = x - x_{\text{hat}}$. A natural distortion objective is mean squared error: $\text{MSE} = \text{mathbbE}[(x - x_{\text{hat}})^2]$.
15. Per-tensor, per-channel, and group-wise quantization	Let a weight matrix be W in $R^m \times n$. Per-tensor quantization Use one scale for the whole tensor: $\text{wide}W_{\text{hat}} = s \cdot \text{round}(W/s)$.
16. Solved numeric examples for scales, clipping, and granularity	These examples are meant to be worked out on the board in front of students. Example 1: symmetric 4-bit quantization Take $x = [-1.8, -0.5, 0.3, 1.2]$.
17. Quantized linear layers	If both W and x are quantized, we may write $W \approx s_W Q_W,; x \approx s_x Q_x$, where Q_W and Q_x are integer tensors. Then the linear map becomes approximately $y \approx s_W s_x (Q_W Q_x) + b$.
18. Part III. Pre-quantization	This notebook heading introduces part iii. pre-quantization and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
19. Pre-quantization means designing the approximation before applying it	The phrase pre-quantization in this chapter means all the work done before committing to a low-bit representation: - dataset or calibration subset selection - observer choice - range estimation - outlier analysis - sensitivity analysis - deciding between per-tensor, per-channel, and group-wise schemes - deciding whether to quantize activations, weights, KV cache, embeddings, or only selected layers - deciding which layers remain in higher precision This phase is conceptually distinct from both QAT and.
20. Calibration as distribution estimation	Suppose an activation tensor a has unknown deployment distribution $p(a)$. Calibration uses a small dataset C to estimate statistics such as: - minimum and maximum - mean and variance - selected percentiles - channel-wise maxima - timestep-dependent ranges in diffusion models - token-position-dependent or layer-dependent outliers in transformers A simple min-max scale for symmetric quantization is $s = \frac{1}{\max_a \text{in } C a Q}$.

21. Sensitivity analysis	Not all layers are equally sensitive to quantization. Let the network be $f(x; W_1, \dots, W_L)$, and suppose only layer ℓ is quantized: $\widehat{W}_{\ell} = W_{\ell} + \Delta W_{\ell}$.
22. What is the Hessian?	The gradient tells us the slope of the loss with respect to each parameter. The Hessian tells us how that slope itself changes. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
23. Using MNIST for Examples	The runnable benchmark is MNIST-first. The loading order is: 1. use local cached MNIST files if they already exist, 2. try TensorFlow/Keras MNIST if available, 3. try torchvision.datasets.MNIST if available, 4. try a small manual downloader and IDX parser, 5. fall back to the sklearn digits dataset only if MNIST cannot be reached in the current environment. The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.
24. A baseline model for experiments	We use a small MLP because it trains quickly and makes the numerical effects of quantization easy to show. The point is not that MLPs are state of the art, but that they let us measure: - accuracy - approximate storage - latency - degradation under different bit-widths The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.
25. Scalar quantization example	Before quantizing a whole network, it is useful to show the simplest scalar example. This example makes two ideas visually obvious: 1. lower bit-width means fewer representable levels, 2. quantization error usually increases as the number of bits decreases. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
26. Pre-quantization algorithm: simple observer-based calibration	A minimal observer-based calibration algorithm is: Algorithm: Min-max observer calibration Input: calibration tensors $a^{(i)}_{i=1}^N$, bit-width b 1. Initialize running min and max: $m_{\min} = +\infty$, $m_{\max} = -\infty$ 2.
27. Pre-quantization Summary	At the end of pre-quantization, we want explicit answers to these questions: - Which tensors are quantized? This asks whether quantization is applied to weights, activations, embeddings, KV cache, nonlinear ops, or other parts of the computation graph. - What bit-width does each tensor get?
28. Conceptual relationships within pre-quantization	This section can be understood as a sequence of decisions made before quantized deployment or quantized training begins. The big picture Pre-quantization asks the following question: Given a trained floating-point model, what information should we collect before quantizing it?
29. Part IV. During-training quantization	This notebook heading introduces part iv. during-training quantization and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
30. Why training-time quantization is hard	The challenge is that quantization includes rounding: $q = \text{round}(x/s)$, and the round function is not differentiable in the ordinary sense. Its derivative is zero almost everywhere and undefined at discontinuities, so naive backpropagation fails.
31. The straight-through estimator (STE)	In the forward pass, we use $\widehat{x}$. In the backward pass, the STE replaces the derivative by something simple, often $\partial \widehat{x} / \partial x \approx 1$ inside the unclipped region.
32. QAT objective	Suppose the full-precision network is $f(x; W)$ and the fake-quantized network is $f_Q(x; W, s)$. Here: - x is the input, - y is the ground-truth target associated with x , - W denotes the trainable network weights, - s denotes quantization parameters such as scales or step sizes, - $\ell(y_{\widehat{}}, y)$ is the loss function comparing the model prediction $y_{\widehat{}}$ to the true target y .
33. PACT: learned clipping during training	A central problem in activation quantization is that activations can occasionally take very large values. If we quantize over a very wide range, then the quantization grid becomes coarse and most values are represented poorly.

34. LSQ: learned step size quantization	LSQ stands for Learned Step Size Quantization. The core idea is simple: > instead of manually fixing the spacing between quantization levels, let training learn that spacing.
35. Binary and ternary training	BinaryConnect and related methods constrain weights to a binary set such as $\widehat{W} \in \{-1, +1\}$. A common approximation is $\widehat{W} \approx \alpha \text{sign}(W)$, where α is a scaling factor.
36. Generic QAT Algorithm with STE	Input: training set D , model parameters W , quantizer parameters θ_Q For each minibatch (x, y) : 1. Fake-quantize weights and activations in the forward pass: $\widehat{W} = Q(W; \theta_Q)$; $\widehat{a} = Q(a; \theta_Q)$ 2.
37. Runnable fake-quant QAT example in PyTorch	The following implementation uses an STE-style fake quantizer. It is intentionally simple so the logic is visible during lecture. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
38. Conceptual relationships within during-training quantization	This section studies what changes once quantization is inserted inside the training process. The big picture During-training quantization asks: How can we optimize model parameters while the forward pass behaves as if low precision will be used?
39. Part V. Post-training quantization	This notebook heading introduces part v. post-training quantization and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
40. Post-training quantization as loss-preserving discretization	In PTQ, the weights are already trained. We do not revisit the original full training loop.
41. Baseline PTQ: round-to-nearest (RTN)	The simplest PTQ method is: $\widehat{W} = s \cdot \text{round}(W/s)$. This is often called round-to-nearest.
42. PTQ on the trained model	We now benchmark a family of post-training methods on the same trained baseline model. The runnable comparison includes: - per-tensor PTQ, where each weight tensor gets one scale, - per-channel PTQ, where each output channel gets its own scale, - dynamic INT8 quantization in PyTorch for linear layers. The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.
43. Dynamic quantization in PyTorch	Dynamic quantization leaves activations in floating point until runtime, where they are quantized on the fly for selected operations such as linear layers. This is attractive for transformer-like and MLP-heavy models because it is easy to apply and often reduces memory substantially. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
44. AdaRound: layerwise rounding optimized using calibration data	Ordinary post-training quantization often uses nearest rounding: $\widehat{w} = s \cdot \text{round}(w/s)$. This is simple, but it makes each rounding decision independently.
45. One-layer notation	Consider one layer with input activation vector $a_{\text{ell-1}}$, weight matrix W_{ell} , bias b_{ell} , and activation function ϕ . The full-precision layer output is $z_{\text{ell}} = W_{\text{ell}} a_{\text{ell-1}} + b_{\text{ell}}$, $a_{\text{ell}} = \phi(z_{\text{ell}})$.
46. Reconstruction objective for one layer	Using the single-example notation, a natural objective is $\min_{\widehat{W}} \ a_{\text{ell}} - \widehat{a}_{\text{ell}}\ _2^2 + \lambda R(\widehat{W})$. Using a calibration batch, the same idea becomes $\min_{\widehat{W}} \ A_{\text{ell}} - \widehat{A}_{\text{ell}}\ _F^2 + \lambda R(\widehat{W})$, where $\ \cdot\ _F$ is the Frobenius norm.

47. Why nearest rounding can be suboptimal	Nearest rounding treats each weight separately: $w_{\hat{i}} = s \cdot \text{round}(w_i/s)$. But a layer output depends on all weights together through the matrix-vector product $W_{\text{ell}} a_{\text{ell}}$.
48. One-neuron example	Consider a single linear neuron with no bias and no nonlinearity for simplicity: $a_{\text{ell}} = W a_{\text{ell}}$. Ordinary nearest rounding Since both weights are closer to 0 than to 1, nearest rounding gives $w_{\text{hat_RTN}} = [0,0]$.
49. Why this sits between RTN and QAT	AdaRound is more powerful than naive round-to-nearest because it uses calibration data and layer reconstruction. But it is cheaper than full quantization-aware training because it does not retrain the whole network end to end.
50. Second-order PTQ and GPTQ	Why curvature matters in post-training quantization To understand second-order post-training quantization, we need to understand the idea of curvature. In optimization, the first derivative tells us the local slope of a function: whether moving a little to the left or right will increase or decrease the loss.
51. Simplified GPTQ-style procedure	Algorithm: blockwise second-order PTQ For each weight matrix W : 1. Collect calibration activations X 2.
52. SmoothQuant: moving difficulty from activations to weights	Transformers and LLMs often contain activation outliers, meaning that a few activation channels can become much larger than the others. This makes full weight-and-activation quantization difficult, especially in the W8A8 (8 bits for weights and activations) setting, because large activation ranges force the activation quantizer to use a coarse grid.
53. Activation-aware Weight Quantization (AWQ)	Weight Quantization means the method primarily quantizes the model weights. - Activation-aware means it uses activation statistics from calibration data to identify which weights are most important to preserve. AWQ is a weight-only LLM PTQ method based on the observation that a small subset of channels or weights are especially important.
54. OmniQuant: learnable PTQ for LLMs	OmniQuant stays in the post-training quantization regime, but it makes the calibration stage significantly more expressive than classical fixed-parameter PTQ. Instead of treating clipping thresholds, migration strengths, or equivalent scaling choices as hand-designed constants, OmniQuant learns additional quantization parameters during calibration in a differentiable block-wise reconstruction framework.
55. Integer-only transformer inference and I-BERT	For real deployment, it is not enough to quantize only the matrix multiplications if the rest of the computation graph still falls back to floating-point arithmetic. In that case, the model is only partially quantized: integer kernels may accelerate some linear layers, but the system still pays the cost of dequantization, floating-point execution, and re-quantization around the nonlinear operations.
56. Summary table across runnable examples	The table below consolidates the MNIST-first benchmark into a single view. It combines: - the floating-point baseline, - post-training quantization results, - dynamic INT8 quantization, - and QAT results. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
57. Effectiveness summary from the runnable experiments	Interpret the benchmark in the following order. Accuracy retention Compare the column accuracy_drop_pct_points. - A small value means the quantized model behaves close to FP32 on the benchmark. - PTQ often performs well at 8 bits and degrades more noticeably at 4 bits or below. - QAT often recovers part of the loss caused by aggressive low-bit quantization. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
58. Representative effectiveness claims from the literature	These are paper-reported headline results that are useful for classroom discussion. They are not directly comparable across tasks or hardware, but they give a realistic sense of what each family can achieve.

59. Conceptual relationships within post-training quantization	This section asks what can be done after a floating-point model has already been trained. The big picture Post-training quantization asks: How can we reduce precision without retraining the full model?
60. Part VI. Quantization for generative models	This notebook heading introduces part vi. quantization for generative models and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
61. Why generative models are harder than classifiers	For a classifier, one forward pass produces a prediction. For many generative models, the network is used repeatedly: - autoregressive decoding in LLMs - iterative denoising in diffusion models - multi-step ODE integration in flow matching or CNFs This means quantization error can accumulate over time rather than affecting only one prediction.
62. LLM quantization	For an autoregressive transformer, each layer repeatedly computes projections such as $Q = X W_Q$; $K = X W_K$; $V = X W_V$. Quantization affects: - projection matrices - MLP matrices - activations - KV cache - sometimes embeddings and output head Main mathematical issues 1.
63. QLoRA and quantized fine-tuning	QLoRA does not mainly target inference-only PTQ. Instead, it supports efficient fine-tuning of a quantized base model.
64. Diffusion model quantization	A diffusion model repeatedly predicts noise or velocity at many timesteps. A simplified reverse update is $x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t \epsilon$.
65. Q-Diffusion and PTQD	Q-Diffusion showed that ordinary PTQ methods do not transfer directly to diffusion models and introduced two important ideas: - timestep-aware calibration - split shortcut quantization PTQD went further by explicitly modeling quantization noise in the denoising process and correcting mean and variance mismatches across timesteps. A high-level way to express the issue is: $\widehat{\epsilon}_{\theta}(x_t, t) = \epsilon_{\theta}(x_t, t) + \eta_t$, where η_t is quantization noise.
66. Diffusion Transformers and DiT-specific PTQ	For DiTs, the same transformer-specific issues seen in LLMs appear together with timestep dependence from diffusion. Hence there are two interacting difficulties: 1. transformer outliers and channel imbalance, 2. timestep-varying distributions.
67. Flow matching and continuous-time generative models	In flow matching, we learn a time-dependent vector field $v_{\theta}(x, t)$ and solve an ODE such as $\frac{dx_t}{dt} = v_{\theta}(x_t, t)$. If we quantize the model, then the deployed vector field becomes $\widehat{v}_{\theta}(x, t) = v_{\theta}(x, t) + e(x, t)$.
68. Algorithmic template for quantizing generative models	Identify the repeated state update: - autoregressive token update for LLMs - denoising step for diffusion - ODE step for flow matching 2. Build calibration data that reflects deployment states: - representative prompts or hidden states for LLMs - representative timesteps and latent states for diffusion - representative time-state pairs for flow matching 3.
69. Why Quantization Is Difficult in Generative and Multimodal Model Families	Summary table: why pre-, during-, and post-quantization are difficult Model family; Why pre-quantization is hard; Why during-training quantization is hard; Why post-training quantization is hard Autoregressive transformers / LLMs; The model contains several numerically different objects such as weights, activations, attention projections, residual streams, normalization layers, and KV cache, so choosing bit-widths, scales, and granularity is already nontrivial. Activation outliers further make scale.
70. Methods by phase and model family	Model family; Pre-quantization methods / ideas; During-training methods / ideas; Post-training methods / ideas Autoregressive transformers / LLMs; Per-tensor / per-channel / group-wise scaling, outlier analysis, mixed precision planning, full-graph operator analysis; Fake quantization, STE-based QAT, LSQ-style learned step sizes, adapter-based low-bit fine-tuning such as QLoRA; GPTQ, SmoothQuant, AWQ, OmniQuant, KV-cache quantization, integer-only operator work such as I-BERT Diffusion models (U-Net,.

71. Summary	A useful way to think about the difficulty is: - LLMs are hard mainly because of activation outliers, attention, normalization, and KV-cache. - Diffusion models are hard mainly because quantization error accumulates across denoising steps. - Flow matching models are hard because low-bit perturbations distort a time-dependent velocity field and its solver path. - CLIP-like models are hard because quantization must preserve cross-modal embedding alignment. - Fusion models are hard because different.
72. Modern transformer, LLM, diffusion, and generative-model references	LLM.int8() (NeurIPS 2022) https://arxiv.org/abs/2208.07339 2. SmoothQuant (ICML 2023) https://arxiv.org/abs/2211.10438 4.
73. What Can Be Quantized vs. Modalities vs. Method	What can be quantized; Common quantization methods; Vision / CNN / ViT; NLP / Transformers / LLMs; Audio / Speech; Multimodal / VLMs; Generative models (LLMs, diffusion, flow matching); Notes Weights; PTQ, QAT, per-tensor, per-channel, group-wise, binary, ternary, GPTQ, AWQ; Yes; Yes; Yes; Yes; Yes; Most common target; easiest place to start Activations; PTQ calibration, QAT, fake quantization, per-tensor, per-channel, SmoothQuant-style scaling; Yes; Yes; Yes; Yes; Yes; Harder than weights because.
74. A more conceptual view	Category; Typical methods; Common modalities; Main idea Parameter quantization; PTQ, QAT, GPTQ, AWQ, binary, ternary; all; Compress stored model parameters Activation quantization; calibration-based PTQ, QAT, SmoothQuant, OmniQuant; all; Reduce runtime memory and enable low-precision inference Training-state quantization; quantized gradients, 8-bit optimizers; all during training; Save training memory and bandwidth Graph-level quantization; integer-only inference, I-BERT, mixed precision graph design;.
75. Short interpretation	PTQ is most natural for pretrained weights. - QAT is most natural when activations and nonlinearities are hard to quantize. - GPTQ / AWQ / OmniQuant / SmoothQuant are especially important for transformer and LLM deployment. - Integer-only methods matter when the goal is true hardware-efficient deployment, not just smaller weights. - Mixed precision is often used when some parts of the model are much more sensitive than others.
76. One Approach for Generative Models	This notebook heading introduces one approach for generative models and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.